

Upgrade Report

Bike lanes, traffic signals and signs, sharper motion cues, one identity per vehicle, licence-plate privacy, and faster processing

September 2026

WHAT CHANGED IN THE PIPELINE

perception → ego-motion → depth → **traffic lights, signs and bike lanes** → track features with **flow motion cues, lane-paint relations and one id per vehicle** → exposure outputs and overlay clips with **licence plates blurred**

0.93 bike-lane F1 per second (CLIP alone: 0.78)	40 / 44 traffic-signal states read correctly	0.97 AUC of the new closing ratio (best existing cue: 0.94)	16 / 16 vehicle-id joins checked by eye were correct
--	---	--	---

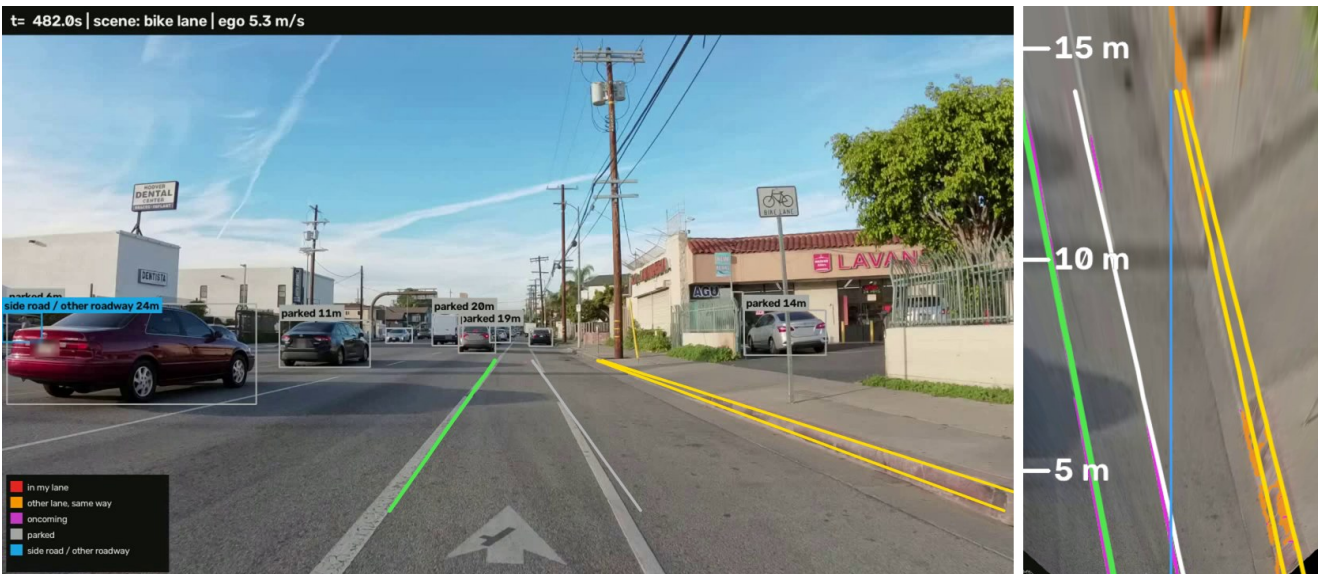


Figure 1. The new bike-lane detector at ride second 482. Right: bird's-eye view of the road 2.5-16 m ahead, where paint strokes (magenta, orange for yellow) are found and fitted as lines; the blue line is the rider's path. Left: the same lines drawn back into the camera view; green marks the line bounding the rider's bike lane. The thin boxes and labels are burned into this demo clip by the earlier overlay. Licence plates are blurred in every figure.

Summary. Bike lanes, traffic signals and stop signs are now detected, reported per second and turned into events. The tracker artefacts that made one vehicle count as several - duplicate boxes on pickups and vans, and tracks broken by a missed detection - are removed, so counts and events are per physical vehicle. Licence plates are blurred in everything rendered; plate reading exists only as an opt-in, hashed identity check. What the new motion features add to the typology accuracy on all 378 labelled tracks is measured by one command on the project workstation (Section 8).

1. Starting point, scope and evidence

The previous milestone typed every vehicle seen from the bike into five relations to the rider (in my lane, other lane same way, oncoming, parked, side road / crossing) and reported exposure per ride. Within 15 m of the rider the selected gradient-boosting model reached 68.1% accuracy and macro-F1 0.503 in leave-one-video-out validation, and the most frequent error was a moving vehicle called parked. The review asked for better accuracy, for bike-lane and traffic-sign detection, and for faster processing; the use of licence plates as vehicle identities was raised during the work.

Completed work

Request	What was built	Evidence
Bike-lane detection	Lane paint and bicycle stencils in a bird's-eye view of the road; a per-second decision fused with the CLIP scene probe	F1 0.78 → 0.93 on 142 labelled seconds (Sec. 2)
Traffic signals	Open-vocabulary detection (YOLOE-26) and lamp-state reading	40 of 44 signal states correct; both real red-light waits found (Sec. 3)
Traffic signs	Open-vocabulary detection; stop signs confirmed by a COCO detector	all 10 false stop signs removed, the real one kept (Sec. 3)
Relation accuracy	Diagnosis; depth-free motion cues from the stored optical flow; lane-paint relations; extra-trees candidate	new cue separates parked from moving traffic best (AUC 0.97 vs 0.94); corpus gain from the ablation (Sec. 4)
Vehicle identity	Duplicate boxes merged, broken tracks joined, one relation per vehicle	16 of 16 checked joins correct; per-second counts and events corrected (Sec. 5)
Plate privacy	Plates blurred by default; opt-in hashed identity audit	all demo videos and snapshots blurred; audit agrees with the vehicle ids (Sec. 6)
Speed	FP16 on the GPU, hardware video decoding, parallel CPU stages, resumable stages	stage timings printed by every run (Sec. 7)

Pipeline after the upgrade

Stage	Module	In this upgrade
Perception (GPU, ~12 fps)	bikesafe.perceive	FP16, hardware decoding, finished videos skipped
Ego-motion (CPU)	bikesafe.egomotion	several rides at once
Depth and lane structure (GPU, 2 fps)	bikesafe.scene3d	FP16; optional (--skip-depth)
Lights, signs, bike lanes (GPU, 2 fps)	bikesafe.infrastructure	new
Track features (CPU)	bikesafe.tracks, .geometry, .stitch	new flow cues, lane-paint relations, vehicle ids
Licence plates	bikesafe.plates	new blurring; opt-in hashed audit
Learning and evaluation	bikesafe.train	extra trees, feature ablation, scene scoring with paint
Outputs	bikesafe.exposure, .render	per-vehicle counts and events; bike lane, signals, signs
One-command runner	bikesafe.run	resumable, parallel, per-stage timings

Evidence available for this review. The ride videos (several GB each) and the intermediate files stay on the project workstation. The upgrades were therefore validated on the three demo clips in the repository (2.5 minutes of real riding, re-run through the whole pipeline), on the approved 300-frame tracking reference, and on the per-track tables of all six rides (14,770 tracks). Numbers that need the full videos - what the new features add on all labels, and corpus-wide counts - are produced by the commands in Section 8, and rebuilding this report then adds them.

2. Bike-lane detection

The CLIP scene probe separated paths from roads well but recognised painted bike lanes poorly (bike-lane F1 0.49 on the labelled scene frames). A bike lane is defined by its paint, so the new detector (`bikesafe.infrastructure`) looks for the paint.

- **Bird's-eye view.** The self-calibrated flat-road geometry the pipeline already uses (horizon, camera height, lateral scale) warps the road 2.5-18 m ahead into a metric top view at 2 cm per pixel, twice per second, with vehicles masked out. Lane lines run vertically there.
- **Paint.** Thin (at most 0.35 m), elongated strokes brighter than the asphalt beside them; kerbs, crosswalk bars, cracks and shadow edges are rejected by width and length. Yellow and white are told apart by saturation, which also works for faded yellow and white in low sun.
- **Lines.** Extracted greedily - the best-supported line, then the paint it explains is removed - each with its own slant, and classified solid or broken, white or yellow.
- **Stencils.** Bicycle symbols are found by YOLOE-26 in the top view, where they are upright and unforeshortened.
- **Decision.** Transparent evidence rules - for: a solid white line 0.3-2.4 m to the left, a narrow lane between two lines, a stencil in the rider's corridor, green paint; against: a 3 m+ traffic lane between broken lines, a yellow centre line beside the rider - averaged over 7 s and held through junctions without paint.
- **Fusion.** CLIP keeps the separated-path call it gets right; the paint decides bike lane versus shared road wherever it has evidence, and CLIP fills the rest.

Per-second check

Riding context per second	Seconds decided	Accuracy	Bike-lane precision	Bike-lane recall	F1
CLIP scene probe (before)	100%	89.4%	100.0%	64.3%	0.78
Lane paint alone	83%	94.9%	97.4%	88.1%	0.93
Fused: paint decides, CLIP fills in (new default)	100%	95.8%	97.4%	88.1%	0.93

142 seconds of the three demo clips labelled by hand from the road view (junction crossings excluded): a boulevard section, a bike pocket before an intersection and a long kerbside bike lane beside parked cars, plus two shared residential streets.

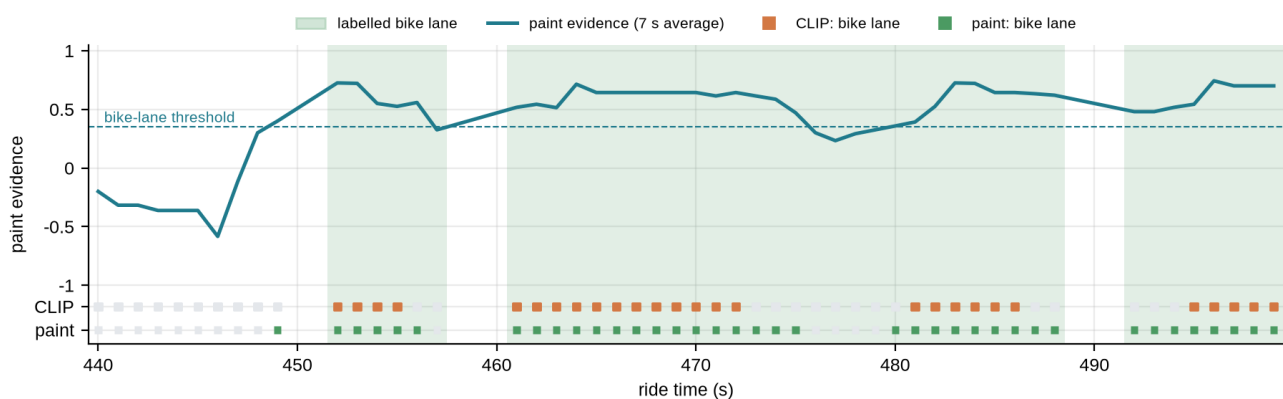


Figure 2. The bike-lane clip second by second. CLIP (orange squares) misses most of the kerbside bike lane; the paint evidence (line) crosses the threshold for nearly all of it. Hollow squares are seconds without usable paint, where CLIP's call is kept.

Caveat. The rules were tuned while looking at these clips, so the table is optimistic. `bikesafe.train` now also scores the labelled scene frames of all six rides with and without the paint (`scene_confusion_probe_plus_lane_paint.csv`), which gives the unbiased number. Night riding, where only the headlight shows the paint, is untested.

3. Traffic signals and signs

- **Detection.** YOLOE-26 (open-vocabulary detection) receives a road-furniture vocabulary - traffic light, stop, yield, speed-limit, street-name, one-way, no-parking and other signs - plus distractor classes (billboard, shop sign, licence plate, street lamp, vehicles, people) that absorb look-alikes. The prompts' text features are cached, so the 240 MB text encoder is not needed at run time.
- **Signal state.** The lit lamp is a bright, saturated, round blob inside the housing (lit reds look pink-magenta on this camera, LED greens cyan-green); sky and walls seen through side-on heads are rejected by shape and position.
- **Stop signs** are confirmed by the COCO-trained YOLO11m already in the repository: the open vocabulary alone also calls DO NOT ENTER, NO PARKING and red shop signs stop signs.
- **One object, one row.** Detections of one sign or signal head are linked across samples (a static object moves straight away from the focus of expansion and grows), giving signs.csv.
- **Events.** stop_sign for each distinct stop sign passed close enough; red_light_wait for a stop of 3 s or more with a red signal read for at least 2 s of it, defined by the stop so that a bus crossing in front of the signal does not split one wait into several.

Check on the demo clips	Result
Signal state, labelled signal-head crops	40 of 44 correct (14 of 16 lit heads; the misses are a tiny far head and a faint side-on one, and two unlit heads read as yellow against a yellow wall)
Signal heads, spot check of 24 detections	23 are signal heads (one teal awning is not)
Stop signs, open vocabulary alone	12 detections, 2 of them stop signs
Stop signs, after COCO confirmation	the real stop sign facing the rider kept, all 10 false ones removed (a small side-on stop sign facing cross traffic is also dropped)
Red-light waits	2 found, both real; one lasted 23 s with a bus crossing in front of the signal
Other signs, 36 detections	20 are the demo clip's own burned-in overlay labels; of the other 16, about 12 are real traffic signs and 4 shop signs



Figure 3. Signal-head crops from the demo clips with the state read by the lamp detector.

Sign types other than stop signs are open-vocabulary guesses (a BIKE LANE sign was called a speed-limit sign), so reports use the number of signs and treat the fine types as indicative.

4. Relation accuracy: where it is lost and what was added

Leave-one-video-out, labelled tracks that came within 15 m: 68.1% accuracy, macro-F1 0.503. The confusion matrix shows the dominant error:

Labelled as	Predicted parked	Labelled tracks	Share
oncoming	12	23	52%
other lane, same way	9	25	36%
side road / crossing	18	68	26%
in my lane	3	11	27%

Three causes were found, each with evidence: the fitted world trajectory drifts for parked cars because the box centre slides outward as a passed car's side comes into view (parked cars end up with a median fitted speed of 2.15 m/s at 95 degrees, like crossing traffic); the depth-model trajectory is biased (parked cars 'move' at +2.1 m/s along the road in it); and vehicles alongside, cut off by the frame edge, lose their geometry.

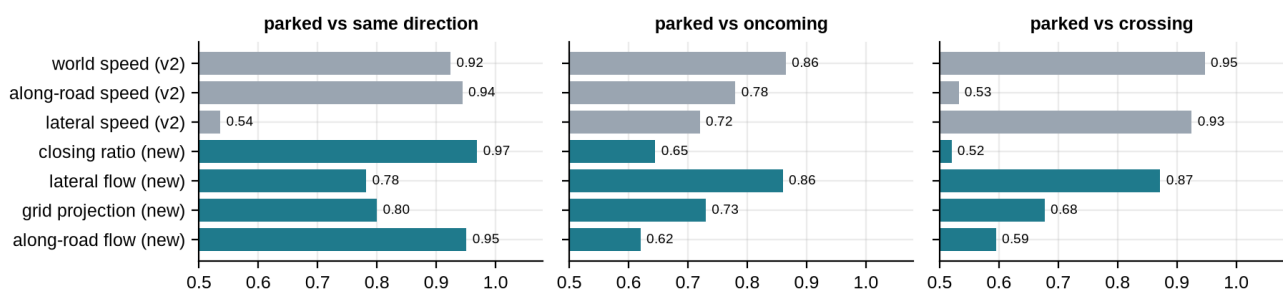
Better modelling of the existing features does not fix this

Model on the existing 42 features	Within 15 m: accuracy / macro-F1	All distances
Gradient boosting (current)	68.1% / 0.503	63.0% / 0.432
Extra trees, balanced (mean of 5 seeds)	68.3% / 0.510	65.9% / 0.482
Random forest	66.8% / 0.419	65.6% / 0.403
Logistic regression	61.0% / 0.475	55.6% / 0.428
+ 16 engineered feature combinations	within ± 2 points	within ± 2 points

With about 300 labelled tracks the standard error of an accuracy is about 2.7 points, so none of these differences is real within 15 m; extra trees were kept as a candidate because they are better over all distances on every seed. The limit is the information in the features, so new information was added.

New information: depth-free motion from the stored optical flow

- **Closing ratio** - the body's measured expansion over the expansion a static object at that distance would show: about 1 for a parked car, 0 for a vehicle moving with the rider, 2 for oncoming traffic at the rider's speed, whatever the absolute scale.
- **Lateral flow** - the body's horizontal image motion minus that of the ground just below it; the rider's own translation and yaw cancel, and box height converts it to metres.
- **Grid projection** - the body's motion against the background beside it, which also works for boxes cut off by the frame edge.
- **Lane-paint relations** - painted lines between the rider's path and the vehicle, whether it sits in the rider's lane, and whether the rider was in a bike lane at the time.



AUC on hand-labelled demo tracks (0.5 = no information, 1 = perfect separation)

Figure 4. How well single features separate parked cars from moving traffic on 76 hand-labelled tracks of the demo clips (rider moving). The closing ratio separates parked cars from same-direction traffic better than any existing feature; lateral flow helps against oncoming traffic.

To be measured on the workstation. The per-track tables in the repository carry the version-2 features only; the flow cues need the perception outputs of the full rides. Running the commands in Section 8 writes results/typology/feature_ablation.csv (version-2 features, + flow cues, + lane paint, for each model, leave-one-video-out on the same labels), and rebuilding this report inserts that table here.

5. One identity per physical vehicle

Track ids are continuity hypotheses of the tracker. Two artefacts make one vehicle look like several, and both inflated counts and events:

- **Duplicate boxes.** The detector suppresses overlapping boxes class by class, so a pickup, van or SUV is often boxed twice at the same time - once as 'car' and once as 'truck' or 'bus' - and each box gets its own track.
- **Breaks.** At 12 frames per second a vehicle near the rider moves tens of pixels between frames; a missed detection or a short occlusion makes the tracker start a new id.

Method (bikesafe.stitch)

- Two tracks that coexist with a mean overlap (IoU) of at least 0.7 over at least half of the shorter one are one vehicle.
- For a break, the last detection of a track is extrapolated forward and the first detection of a later track backward to the middle of the gap. The pair is joined when the predictions meet and the image velocities agree ($e + 0.5 dv \leq 0.45$, both relative to box height), the gap is at most 2 s, class and size agree, neither end is at the side edge of the frame, no other vehicle fits about as well, and the first vehicle is gone before the second appears. A one-to-one assignment picks the joins.
- Tracks keep their own features and labels; `vehicle_id` groups them. Each vehicle gets one relation - the detection-weighted vote of its tracks' class probabilities - and counts, per-second exposure and events are per vehicle.

Check on the bike-ride demo clips (the setting the pipeline runs in)

Demo clip	Minutes	Track ids	Duplicate merges	Break joins	Vehicles
006_bikelane_440s	1.00	221	24	1	196
005_following_280s	0.75	111	14	1	96
006_parked_700s	0.75	168	21	2	145
All	2.50	500	59	4	437

Every break join (4) and the 12 duplicate merges with the lowest overlap were checked by eye on the video: all 16 join the same vehicle. The gate was set on 25 candidate pairs labelled by eye beforehand: it accepts 14 of the 19 same-vehicle pairs and 0 of the 6 different-vehicle pairs. An earlier version without duplicate merging made 23 joins, all of them correct, most of them between flickering duplicate boxes - which is how the duplicates were found.

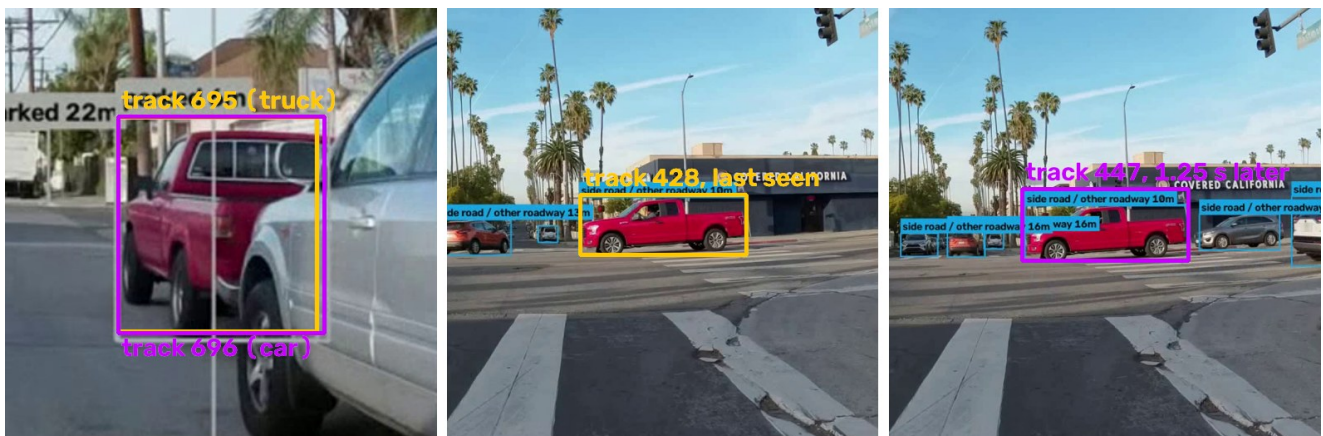


Figure 5. Left: one pickup boxed twice at the same time, as 'truck' (track 695) and 'car' (track 696) - now one vehicle. Middle and right: a pickup crossing ahead loses its track for 1.25 s and comes back as track 447 - joined to track 428.

Effect on the outputs (demo clips, same data before and after)

Measure	Before: per track	After: per vehicle
parked vehicles per minute	81.2	80.8
other lane, same way, per minute	3.2	3.2
side road / crossing per minute	7.6	7.2
events: vehicle close in my lane	9	5
events: crossing traffic near	18	17
events: overtaken by a vehicle	4	4

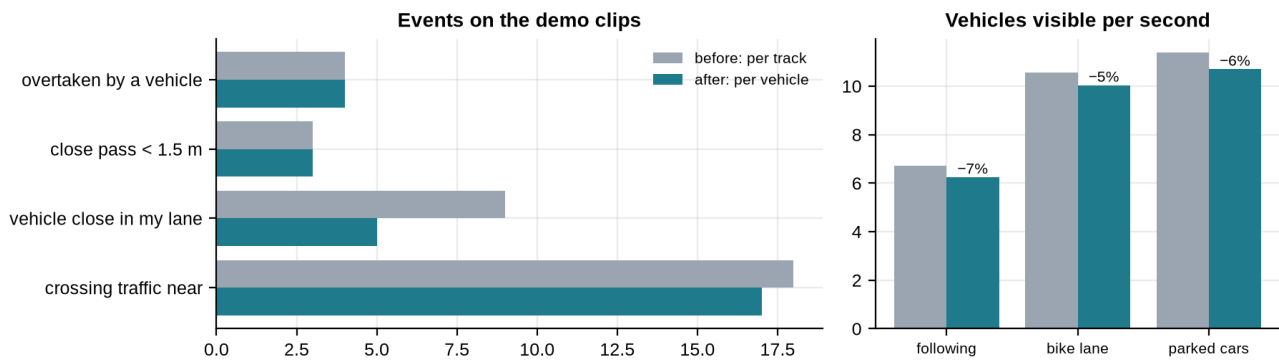


Figure 6. Per-minute rates barely change, because the existing count rule (at least 1 s, within 15 m) already dropped most short duplicates. Events and per-second exposure did not have that protection: 'vehicle close in my lane' fell from 9 to 5 (two duplicate boxes of the car being followed, two short duplicate tracks of parked vehicles), and the vehicles visible per second fell by 5-8%.

Check against the approved 300-frame tracking reference

Setting	IDF1	MOTA	ID switches	Fragments	Reference ids
30 fps	68.4%	81.0%	68	55	89
30 fps + vehicle ids	67.7%	81.4%	46	55	89
30 fps, duplicates removed	68.5%	81.6%	45	57	77
30 fps, duplicates removed + break joins	68.5%	81.6%	45	57	77
15 fps	66.5%	77.5%	36	38	87
15 fps + vehicle ids	66.2%	77.6%	33	38	87
15 fps, duplicates removed	67.0%	78.1%	33	38	75
15 fps, duplicates removed + break joins	67.0%	78.1%	33	38	75

The reference is a 10 s clip of dense, mostly distant traffic from the earlier tracking video. Of its 69 identity breaks, 67 are swaps between two tracks that exist at the same time (all of them on cars under 80 px tall, i.e. more than about 15 m away and outside the exposure counts) and only 2 are a track ending and a new one starting. Break joining therefore makes no join there - and so no wrong one. Merging duplicates cuts identity switches from 68 to 46 at 30 fps, but IDF1 moves by -0.7 points, because the reference itself labels the second ('truck') box of a doubly detected pickup as 12 separate short identities. With those second boxes removed from both sides the scores are unchanged by the joins. This baseline was re-run with the current libraries (IDF1 68.4%; the earlier report measured 72.5% with the same detector and tracker), so it is compared only with itself.

6. Licence plates and privacy

The repository is public, and plates were readable in the rendered demo clips. Two uses of plates were considered: blurring them everywhere, and using them as vehicle identities.

Blurring (default)

- A small plate detector (YOLOv9-tiny, 7 MB ONNX, open-image-models) runs on every vehicle box at least 40 px tall, cropped from the full-resolution frame so a plate is large enough to find; about 24 ms per vehicle on a CPU.
- A plate found on a track stays blurred for 6 more frames at the same place on the vehicle, which covers single missed detections. The blur pixelates and smooths an area 30% larger than the plate.
- The overlay renderer blurs by default; `python -m bikesafe.plates blur` blurs existing videos and images.

File	Frames	Plate regions blurred
demos/typology_005_following_280s.mp4	540	1,403
demos/typology_006_bikelane_440s.mp4	720	2,793
demos/typology_006_parked_700s.mp4	540	1,528
demos/infrastructure_006_bikelane_452s.mp4	480	2,140
demos/best_tracking_demo.mp4	1802	4,012
demos/tracker_visual_comparison.mp4	300	1,409
results/tracking_10s.jpg	1	5
results/tracking_30s.jpg	1	8
results/tracking_50s.jpg	1	8

All demo videos and tracking snapshots in the repository were replaced by blurred versions. The approved 300-frame sequence is benchmark input and was left unchanged.

Should the plate be the vehicle id? No.

Plates are readable only close to the rider and from the front or rear. From the per-track tables of all six rides, at most 31% of the counted tracks ever show a plate 60 px wide or more, and only 16% are also seen mostly from front or rear - an upper bound before blur, glare, night and missing front plates. Crossing and oncoming traffic almost never shows one. An id that exists for a minority, and mostly for parked and same-direction vehicles, would also bias counts between relations. So the id comes from motion and overlap for every vehicle (Section 5), and plates are only an optional check.

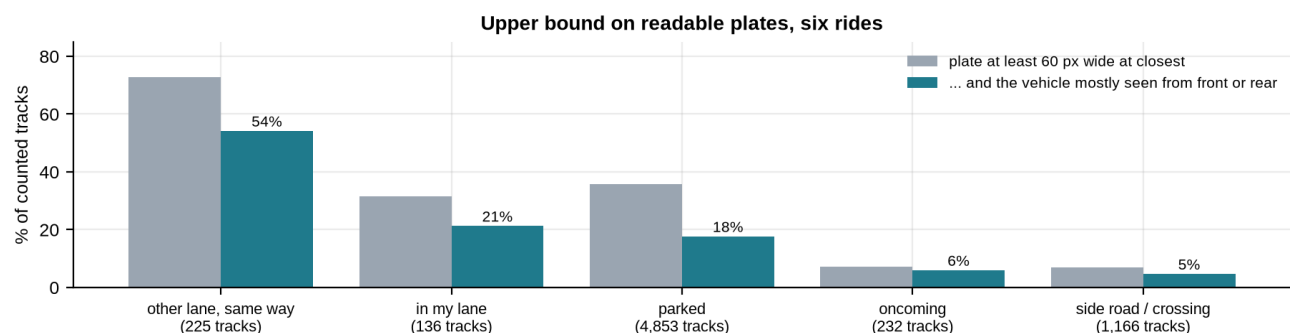


Figure 7. Share of counted tracks (within 15 m, at least 1 s) whose plate could be large enough to read, estimated from each track's largest box (a plate is about a fifth of a car's height).

Identity audit (opt-in, hashed)

- `bikesafe.run --read-plates` reads the plate of each large vehicle on up to 6 frames with a small OCR model (fast-plate-ocr, global model including US plates).
- The text never leaves the process: each read becomes an HMAC-SHA256 hash under a random key created for that run and never stored. Hashes compare only within one ride, cannot be turned back into a plate, and cannot follow a vehicle across rides. They stay in the analysis folder; outputs get counts only.
- A plate counts only if it sits in the lower middle of its own vehicle's box and two reads agree.

- The audit reports tracks with one plate that ended up in different vehicle ids (a missed join, or the vehicle came back later) and vehicle ids carrying two different plates (a wrong join).

On the demo clips - 720p copies with the old overlay burned in, far below the quality of the original footage - a plate was found on 163 of 369 large tracks but read consistently on only 8. For the car the rider followed, the plate hash appears on 6 tracks, all in one vehicle id (15 agreeing pairs, no split, no conflict). The first version of the audit flagged three more tracks with that plate: they were a mail van beside the car, whose crop contained the car's plate - a reader error, fixed by the ownership rule above, not a vehicle-id error.

Limitations. A plate on a vehicle the detector did not box is not blurred. Earlier, unblurred versions of the demo videos remain in the repository's git history; making the repository private, or rewriting its history, is the owner's decision.

7. Faster runs

Change	Effect
FP16 on CUDA for the detector, CLIP and the depth model (default; --fp32 to switch off)	roughly 1.5-2x faster GPU inference
Hardware video decoding (--hw-decode: NVDEC / D3D11 / VA-API through FFMpeg)	decoding of high-bitrate 360 exports moves off the CPU in every stage
CPU stages for several rides at once (--jobs, default half the cores)	ego-motion and track features scale with cores
Every stage resumable; finished videos skipped before any model loads	re-running after a code change runs only what changed
Track features recomputed only when written by an older feature version	vehicle ids and flow cues added without re-running perception
Lights, signs and bike lanes sampled at 2 fps; bird's-eye view at 640 px	the new pass costs a fraction of perception
--skip-depth	drops the depth pass (its features moved scores by -2.3 to +1.6 points)
Per-stage timing summary at the end of bikesafe.run	shows where the time goes on each machine

Measured here (4 CPU cores, no GPU)

Stage	Time
Perception, approved 10 s sequence at 15 processed fps (CPU)	24 s
Lights, signs and bike lanes (YOLOE-26l at 1280 px + bird's-eye view, CPU)	about 2 s per sample
Track features incl. vehicle ids, 2.5 min of demo video	11 s
Vehicle ids alone (bikesafe.stitch)	0.3-0.4 s per minute of video
Exposure outputs, three demo clips	3 s
Plate reading for the audit (opt-in), 2.5 min of video	80 s
Plate blurring in the overlay renderer	about 25 ms per vehicle box

GPU timings depend on the workstation; bikesafe.run prints them per stage.

8. Running the upgrade on the full corpus

On the project workstation, one command reuses everything already computed, runs only the new stages and recomputes the track features; training then measures what the new features add, exposure writes the corrected outputs, and the last command rebuilds this report with the corpus numbers:

```
python -m bikesafe.run D:\rides --out-root D:\out --render-minutes 1 --hw-decode
python -m bikesafe.train --analysis D:\out\analysis --perception D:\out\perception
python -m bikesafe.exposure --analysis D:\out\analysis --perception D:\out\perception --out results\corpus
python scripts\build_upgrade_report.py
```

Add `--read-plates` to the first command for the hashed identity audit.

Limitations and next steps

Limitation	Effect	Next step
Bike-lane rules tuned on the demo clips	optimistic bike-lane scores	score the labelled scene frames of all rides (bikesafe.train does this)
Flow-cue gain not yet measured on all labels	accuracy change unknown	run the commands above; feature_ablation.csv
Sign types beyond stop signs are open-vocabulary guesses	fine sign types indicative only	label a few hundred sign crops and fine-tune
Identity swaps between distant vehicles	not fixed by joins or merges	matter little for exposure (beyond 15 m)
Vehicles that leave and return are new vehicles	leapfrogging counted twice	the opt-in plate audit counts such returns
Night riding	less recall, paint visible only in the headlight	validate on night rides

Conclusion

The pipeline now reports the riding infrastructure the review asked for - painted bike lanes, traffic signals with their state, stop signs and other signs - and turns it into per-second context and events. Counts and events are per physical vehicle, which removed double-counted events that the earlier per-track outputs contained. Licence plates are blurred in everything the project shows, and the decision not to use plates as identities rests on measured coverage. The typology accuracy work identified why moving vehicles were called parked and added depth-free motion cues that separate them well on hand-labelled tracks; their effect on the full labelled set is one command away.

Method references

YOLOE: Real-Time Seeing Anything - <https://arxiv.org/abs/2503.07465>; Ultralytics YOLOE and YOLO11 documentation - <https://docs.ultralytics.com/>

BoT-SORT: Robust Associations Multi-Pedestrian Tracking - <https://arxiv.org/abs/2206.14651>

Depth Anything V2 - <https://arxiv.org/abs/2406.09414>

IDF1: Ristani et al., Performance Measures and a Data Set for Multi-Target, Multi-Camera Tracking - <https://arxiv.org/abs/1609.01775>

open-image-models (plate detector) - <https://github.com/ankandrew/open-image-models>; fast-plate-ocr - <https://github.com/ankandrew/fast-plate-ocr>

HMAC: Keyed-Hashing for Message Authentication, RFC 2104 - <https://www.rfc-editor.org/rfc/rfc2104>